Session 3: Regression and Graph

Regression, Modelsummary table, Graphs

Karry Jiao
dj398@cam.ac.uk
KJ Github

Oct. 2026

Table of contents

1. Introduction
2. Lecture review
3. Correlation
4. Regression
5. Graph plotting
6. Appendix
7. Takeaways

Introduction

What we will learn today

Correlation and regression

- Measure the correlations `cor()`.
- Fit a simple and multiple linear regression with `lm()`.
- Understand the regression outputs; explore regression tables.

Data visualization

- Understand how `ggplot2` works
- Make a scatter plot, histogram, box plot, line chart and pie chart.

Examples use World Happiness Report data.

Open your existing RStudio project and create `exercises-session3.R`.

Lecture Review

Lecture review — From points to a fitted line

A scatter plot shows the observed pairs `(x, y)`.

A simple regression fits the line:

$$\hat{y}_i = \widehat{\beta}_0 + \widehat{\beta}_1 x_i$$

- **Intercept** $\hat{\beta}_0$: predicted Y when X = 0.
- **Slope** $\hat{\beta}_1$: predicted change in Y for a one-unit increase in X.
- **Residual** $e_i = y_i - \hat{y}_i$: observed minus fitted value.

Ordinary least squares (OLS) chooses the coefficients to minimize **the sum of squared residuals**.

Lecture review — Read regression evidence

Consider the illustrative model `earnings ~ education`:

- **Coefficient**: direction and magnitude of the association, in Y units per X unit.
- **Standard error**: estimated uncertainty around the coefficient.
- **p-value**: evidence against the null hypothesis that this coefficient equals zero, under model assumptions.
- **R-squared**: share of variation in Y accounted for by the fitted model in the sample.

Lecture review — Match the graph to the question

What do we want to see?	Basic plot
Association between two numerical variables	Scatter plot
Distribution of one numerical variable	Histogram
Distribution across categories	Box plot
Changes across time	Line chart
Shares of a whole	Pie chart (few categories only)

Always inspect the **axes, units, sample and scales** before drawing conclusions.

Correlation

Load the WHR2016 dataset

```
# Load the 2016 WHR dataset
WHR2016 ← read.csv(
  here("DataWork", "DataSets", "Raw", "Un WHR", "WHR2016.csv")
)

# Clean the variable names
WHR2016 ← clean_names(WHR2016)
```

Calculate correlation: `cor()`

Base R

```
cor(
  WHR2016$happiness_score,
  WHR2016$economy_gdp_per_capita
)
```

- `$` selects one column from a data frame.
- `cor(x, y)` calculates Pearson correlation by default.

Using `%>%` and `with()`

```
WHR2016 %>%
  with(cor(happiness_score,
           economy_gdp_per_capita))
```

Handling missing values

```
WHR2016 %>%
  with(cor(happiness_score,
           economy_gdp_per_capita,
           use = "complete.obs"))
```

- `with()` refers to columns without `$`.
- `complete.obs` keeps rows observed on both variables.

Question: Is the relationship positive or negative? How strong is it?

Correlations for more than two variables

```
# Select three numeric variables
whr_vars ← WHR2016 %>%
  select(happiness_score, economy_gdp_per_capita,
         health_life_expectancy)

# Calculate a correlation matrix
cor(whr_vars, use = "complete.obs")
```

- `select()` keeps only the specified columns.
- `cor(dataframe)` returns a correlation matrix.
- Each cell shows the correlation between two variables.
- The diagonal is always 1, and the matrix is symmetric.

Remember: Correlation measures linear association, not causation.

Running regressions

Linear regression in R: `lm()`

The **base R** function `lm()` estimates linear regression models.

```
lm(function, data = DATA)
```

- `function`: writes the dependent variable and explanatory variables.
- `data`: identifies the data frame containing the variables.

```
# A simple regression: one explanatory variable
reg1 ← lm(happiness_score ~ economy_gdp_per_capita,
          data = WHR2016)
```

- `~` means "model the variable on the left using the variables on the right".

Explore regression output: `summary()`

```
reg1 ← lm(happiness_score ~ economy_gdp_per_capita,
          data = WHR2016)
summary(reg1)
```

- **Estimate:** slope or intercept coefficient.
- **Std. Error:** estimated uncertainty of the coefficient.
- **Pr(>|t|):** p-value testing whether the coefficient equals zero.
- **Multiple R-squared:** fraction of sample variation in happiness explained by the fitted regression.

Question: What does the coefficient on GDP per capita tell us? Does it establish causality?

Multiple linear regression with `lm()`

```
# Add a second explanatory variable
reg2 ← lm(
  happiness_score ~ economy_gdp_per_capita + health_life_expectancy,
  data = WHR2016
)
summary(reg2)
```

- `+` includes another explanatory variable in the model.
- A slope now measures an association **holding the other included variable constant**.
- Compare the GDP coefficient and R-squared across models.

Practice: Fit and compare two models

Exercise 1: Using `WHR2016`, estimate `reg1` and `reg2` from the previous slides. Run `summary()` for each. Has the GDP coefficient changed after adding life expectancy?

```
summary(reg1)
summary(reg2)
```

Remember: An OLS association is not automatically a causal effect.

Comparing regression models

Regression tables with `modelsummary()`

`modelsummary()` produces a readable table from one or more fitted models.

```
# The models were created with lm() on earlier slides
modelsummary(list("Without control variables" = reg1,
                  "With control variables" = reg2))
```

- `list()` groups models for side-by-side comparison.
- Model names become the table's column headings.
- Compare coefficient estimates, standard errors and model fit.

Interpreting a regression table

- **Rows:** explanatory variables and estimated coefficients.
- **Columns:** different model specifications.
- **Standard errors:** uncertainty accompanying coefficient estimates.
- **Goodness of fit:** statistics such as N and R-squared.

> Compare the GDP per capita coefficient between the simple and multiple models. What might explain any difference?

Advanced table formatting and export options are deliberately reserved for a later session.

Graph plotting in R

Data for plotting

Load the original `whr_panel` dataset and packages (`ggplot2`, `ggpubr`, `ggrepel`, `ggthemes`).

```
# load whr_panel
whr_panel ← read_csv(here("DataWork","DataSets","Final","whr_panel.csv"))

library(ggplot2)
library(ggpubr)
library(ggrepel)
library(ggthemes)
```

Purpose

In our workflow there are usually two distinct uses for plots:

1. **Exploratory analysis**: Quickly visualize your data in an insightful way.
    - Base R can be used to quickly create basic figures
    - We will also use `ggplot2` to quickly create basic figures as well.
2. **Publication/Reporting**: Make pretty graphs for a presentation, a project report, or papers:
    - We'll do this using `ggplot2` with more customization. The idea is to create beautiful graphs.

Exploratory Analysis

Plot with Base R

First, we're going to use base plot, i.e., using Base R default libraries. It is easy to use and can produce useful graphs with very few lines of code.

Exercise 2: Exploratory Analysis.

(1) Create a vector called `vars` with the strings: `"economy_gdp_per_capita"`, `"happiness_score"`, `"health_life_expectancy"`, and `"freedom"`.
(2) Select all the variables from the vector `vars` in the `whr_panel` dataset and assign to the object `whr_plot`. Hint: use `select(all_of(vars))` for this.
(3) Use the `plot()` function: `plot(whr_plot)`

1-minute exercise

Plot with Base R

First, we're going to use base plot, i.e., using Base R default libraries. It is easy to use and can produce useful graphs with very few lines of code.

Exercise 2: Exploratory Analysis.

(1) Create a vector called `vars` with the strings: `"economy_gdp_per_capita"`, `"happiness_score"`, `"health_life_expectancy"`, and `"freedom"`.
(2) Select all the variables from the vector `vars` in the `whr_panel` dataset and assign to the object `whr_plot`. Hint: use `select(all_of(vars))` for this.
(3) Use the `plot()` function: `plot(whr_plot)`

1-minute exercise

```
# Vector of variables
vars ← c("economy_gdp_per_capita", "happiness_score", "health_life_expectancy", "freedom")
# Create a subset with only those variables, let's call this subset whr_plot
whr_plot ← whr_panel %>%
  select(all_of(vars))
```

Base Plot

```
plot(whr_plot)
```

The beauty of ggplot2

1. Consistency with the Grammar of Graphics
    - This book is the foundation of several data viz applications: `ggplot2`, `polaris-tableau`, `vega-lite`
2. Flexibility
3. Layering and theme customization
4. Community

It is a powerful and easy to use tool (once you understand its logic) that produces complex and multifaceted plots.

The Structure of ggplot2

Creating a plot with ggplot2 requires **three basic components**:

1. **Data**: The dataset you want to visualize.
2. **Aesthetics (aes)**: How you map your data to visual elements (e.g., x-axis, y-axis, color).
3. **Geometry (geom)**: The type of plot you want (e.g., bar graph, scatter plot).

(and many more, we will see some examples later)

ggplot2: Basic Structure

The basic ggplot structure is:

```
ggplot(data = DATA)
  GEOM_FUNCTION(mapping = aes(AESTHETIC MAPPINGS))
```

Mapping data to aesthetics

Think about colors, sizes, x and y references

ggplot2: Full Structure

The full structure of a `ggplot2` visualization follows a logical sequence.

```
ggplot(data = <DATA>) +
  <GEOM_FUNCTION>(
    mapping = aes(<MAPPINGS>),
    stat = <STAT>,
    position = <POSITION>
  ) +
  <COORDINATE_FUNCTION>() +  # Adjust coordinate syste
  <FACET_FUNCTION>() +       # Create small multiples
  <SCALE_FUNCTION>() +       # Adjust scales (colors, a
  <THEME_FUNCTION>()         # Customize overall appea
```

Key Components

1. **Data** (`data = <DATA>`) → The dataset you want to visualize
2. **Layers** (`geom_`, `stat_`) → Shapes and statistical summaries
3. **Aesthetics** (`aes()`) → Maps data to visual properties
4. **Scales** (`scale_`) → Defines color, size, axes, etc.
5. **Coordinate System** (`coord_`) → Maps data to a 2D plane
6. **Facets** (`facet_`) → Splits data into small multiples
7. **Themes** (`theme()`, `theme_`) → Customizes visual appearance

ggplot2: decomposition

There are multiple ways to structure plots with ggplot

For this session, I will stick to Thomas Lin Pedersen's decomposition who is one of most prominent developers of the ggplot and gganimate package.

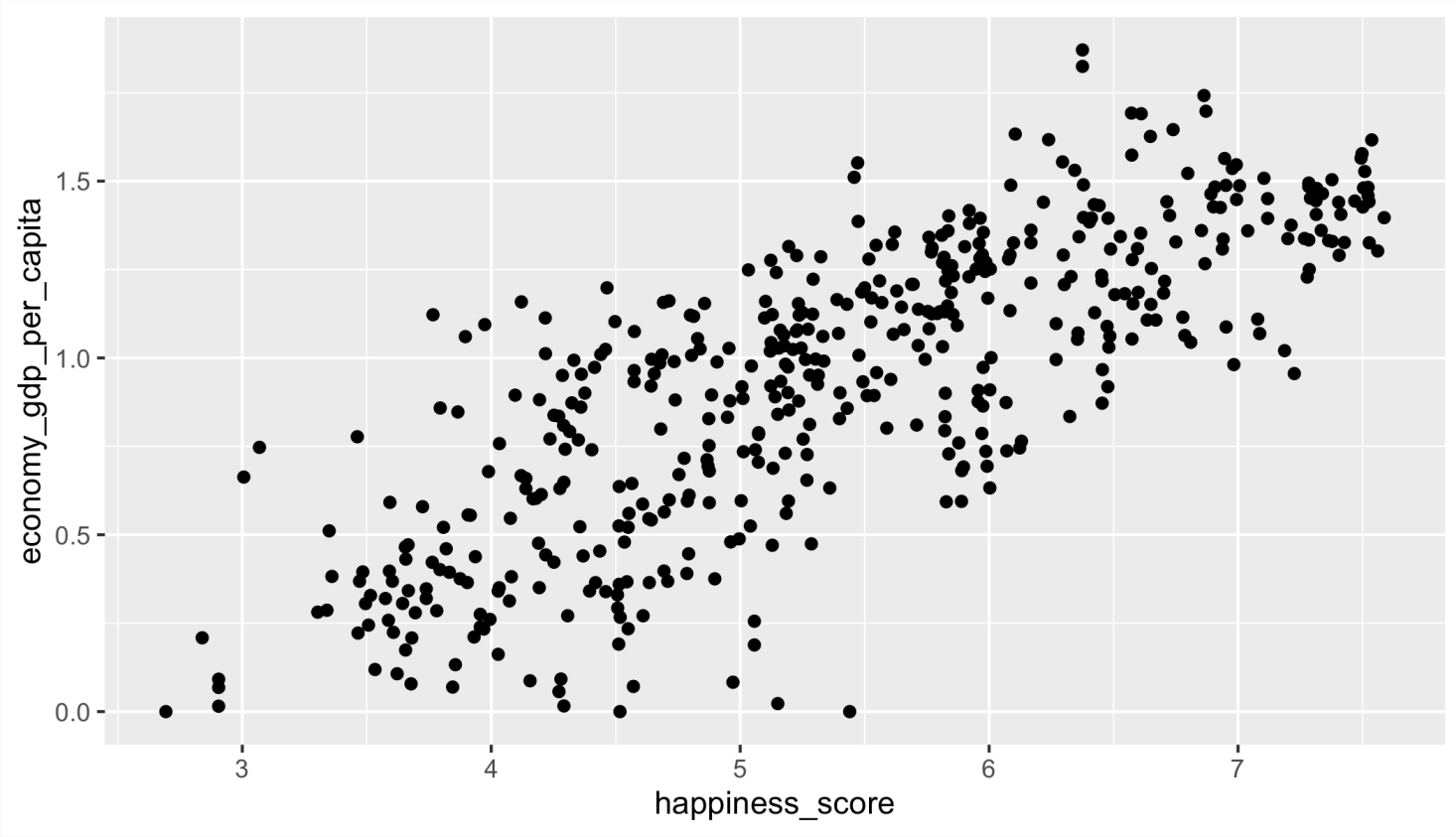
These components can be seen as layers, this is why we use the `+` sign in our ggplot syntax.



Exploratory Analysis

Let's start making some plots.

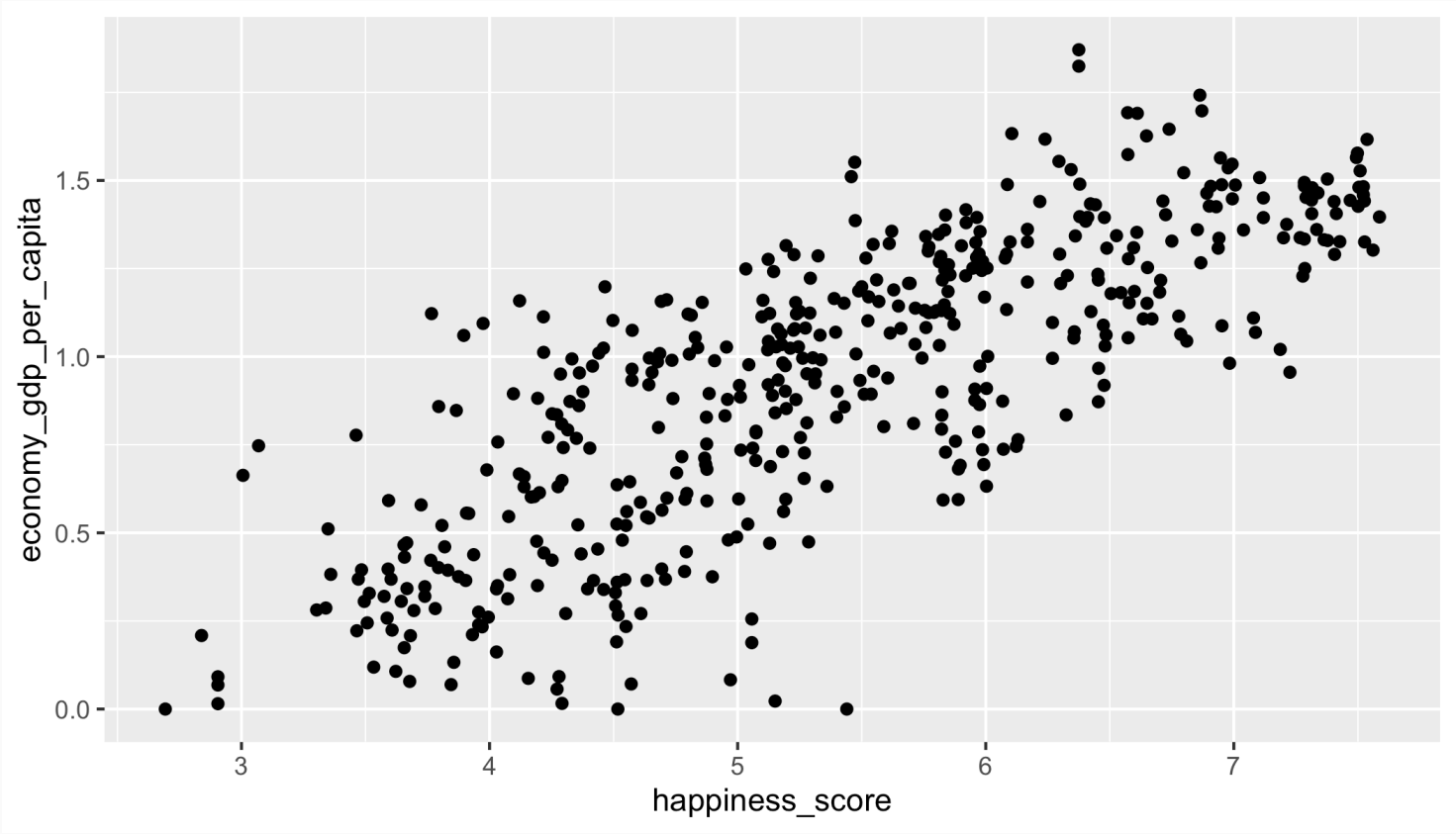
```
ggplot(data = whr_panel) +  #data
  geom_point(aes(x = happiness_score, y = economy_gdp_per_capita))
```



Exploratory Analysis

We can also set up our mapping in the `ggplot()` function.

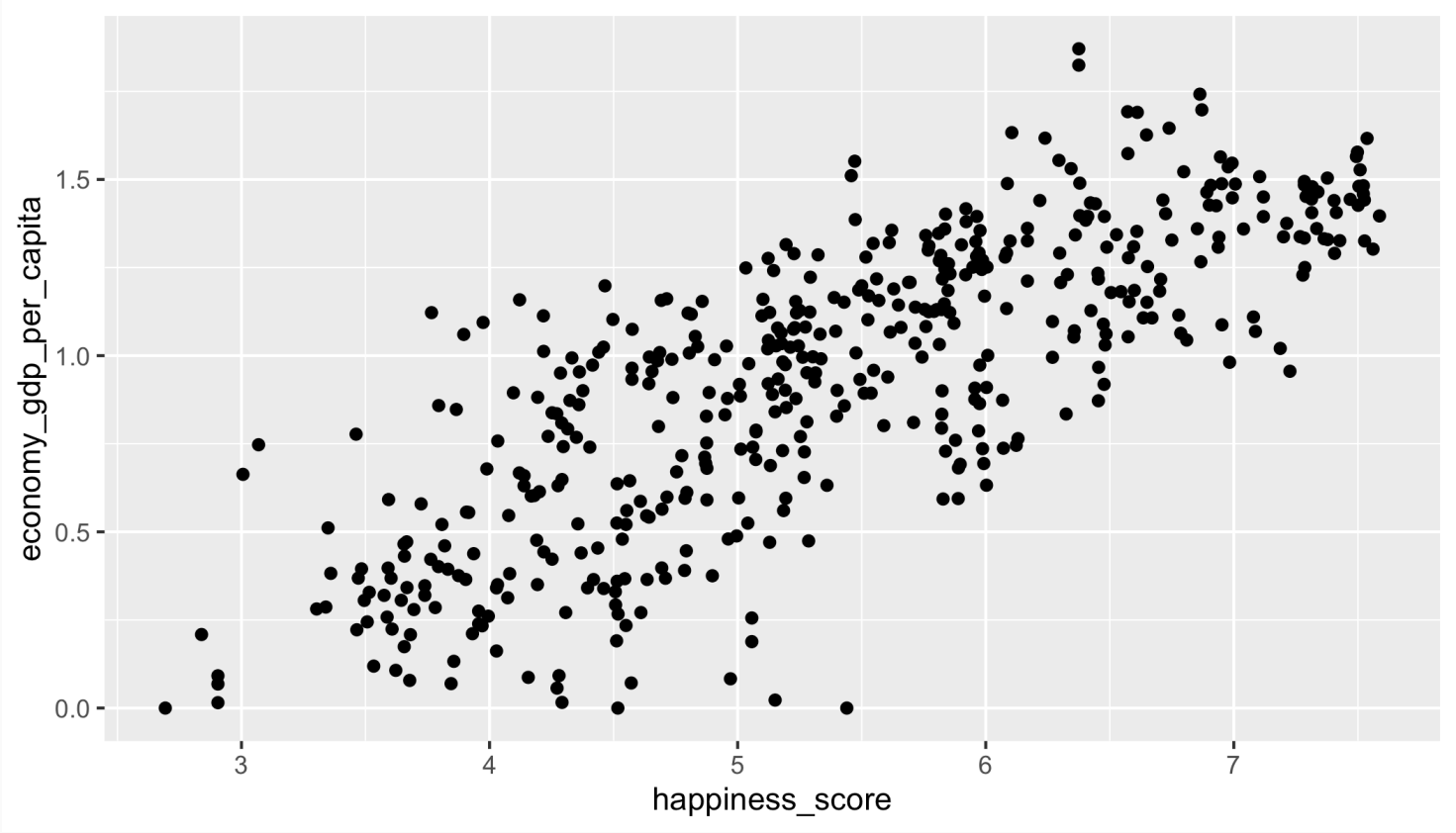
```
ggplot(data = whr_panel, aes(x = happiness_score, y = economy_gdp_per_capita)) +
  geom_point()
```



Exploratory Analysis

We can also set up the data outside the `ggplot()` function as follows:

```
whr_panel %>%
  ggplot(aes(x = happiness_score, y = economy_gdp_per_capita)) +
  geom_point()
```



Exploratory Analysis

The second way of structuring our ggplot looks neat and easy to follow.

1. First, setting our data;
2. pipe it;
3. then aesthetics;
4. and finally the geometries.

Both structures will work but this will make a difference if you want to load more datasets at the same time, and whether you would like to combine more geoms in the same ggplot. More on this in the following slides.

Exploratory Analysis

Exercise 3: Create a scatter plot with `x = freedom` and `y = economy_gdp_per_capita`.

1-minute exercise

Exploratory Analysis

Exercise 3: Create a scatter plot with `x = freedom` and `y = economy_gdp_per_capita`.

1-minute exercise

Solution:

```
whr_panel %>%
  ggplot() +
  geom_point(aes(x = freedom, y = economy_gdp_per_capita))
```

Exploratory Analysis

The most common `geoms` are:

- `geom_bar()`, `geom_col()`: bar charts.
- `geom_boxplot()`: box and whiskers plots.
- `geom_density()`: density estimates.
- `geom_jitter()`: jittered points.
- `geom_line()`: line plots.
- `geom_point()`: scatter plots.

> If you want to know more about layers, you can refer to this.

Exploratory Analysis

In summary, our basic plots should have the following:

```
whr_panel %>%
  ggplot(
    aes(
      x = happiness_score,
      y = economy_gdp_per_capita
    )
  ) +
  geom_point()
```

The data we want to plot.

Exploratory Analysis

In summary, our basic plots should have the following:

```
whr_panel %>%
  ggplot(
    aes(
      x = happiness_score,
      y = economy_gdp_per_capita
    )
  ) +
  geom_point()
```

Columns (variables) to use for `x` and `y`

Exploratory Analysis

In summary, our basic plots should have the following:

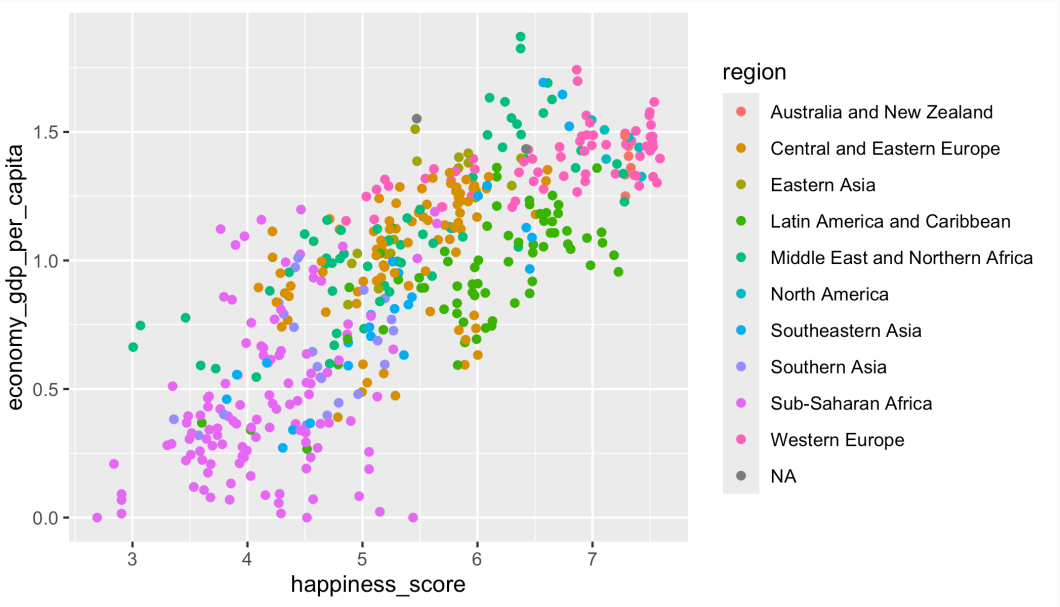
```
whr_panel %>%
  ggplot(
    aes(
      x = happiness_score,
      y = economy_gdp_per_capita
    )
  ) +
  geom_point()
```

How the plot is going to be drawn.

Exploratory Analysis

We can also **map** colors.

```
whr_panel %>%
ggplot(
  aes(
    x = happiness_score,
    y = economy_gdp_per_capita,
    color = region
  )
) +
  geom_point()
```



Exploratory Analysis

Let's try to do something different, try, instead of `region`, adding `color = "blue"` inside `aes()`.

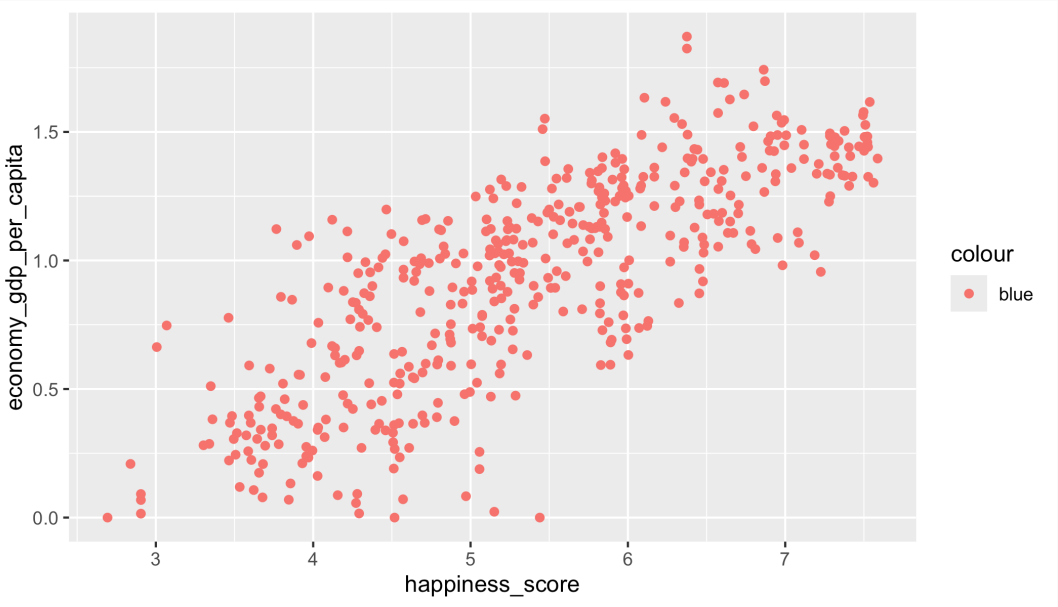
- What do you think is the problem with this code?

Exploratory Analysis

Let's try to do something different, try, instead of `region`, adding `color = "blue"` inside `aes()`.

- What do you think is the problem with this code?

```
whr_panel %>%
  ggplot(
    aes(
      x = happiness_score,
      y = economy_gdp_per_capita,
      color = "blue"
    )
  ) +
  geom_point()
```



Exploratory Analysis

In `ggplot2`, these settings are called **aesthetics**.

> "Aesthetics of the geometric and statistical objects".

We can set up:

- `position`: x, y, xmin, xmax, ymin, ymax, etc.
- `colors`: color and fill.
- `transparency`: alpha.
- `sizes`: size and width.
- `shapes`: shape and linetype.

Notice that it is important to know where we are setting our aesthetics.

- `geom_point(aes(color = region))` to color points based on the variable `region`
- `geom_point(color = "red")` to color all points in the same color.

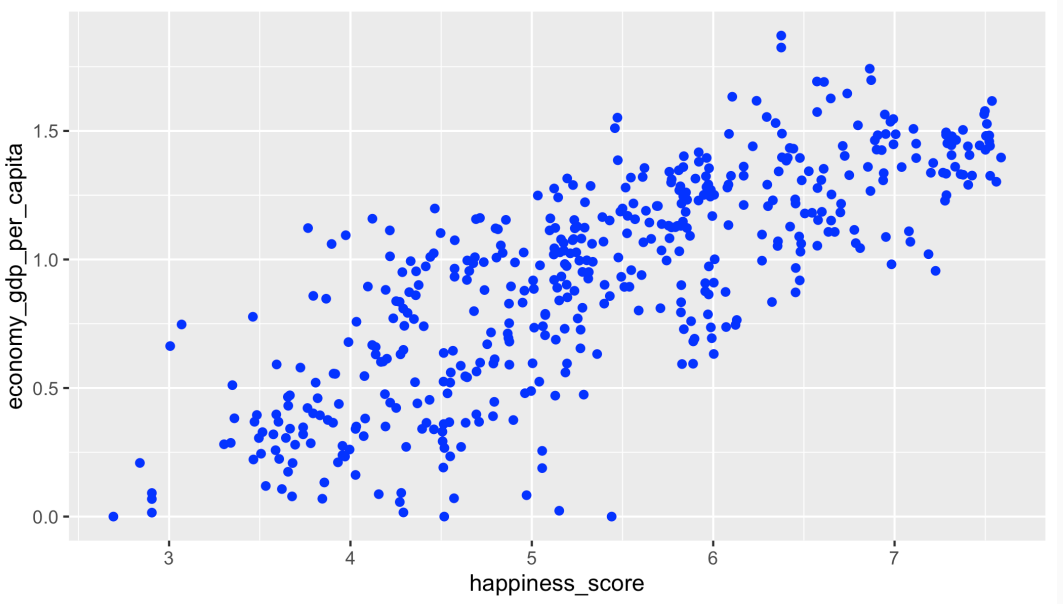
Exploratory Analysis

Let's modify our last plot. Let's add `color = "blue"` inside `geom_point()`.

Exploratory Analysis

Let's modify our last plot. Let's add `color = "blue"` inside `geom_point()`.

```
whr_panel %>%
  ggplot(
    aes(
      x = happiness_score,
      y = economy_gdp_per_capita
    )
  ) +
  geom_point(color = "blue")
```



Exploratory Analysis

Exercise 4: Map colors per year for the freedom and gdp plot we did before. Keep in mind the type of the variable `year`.

1-minute exercise

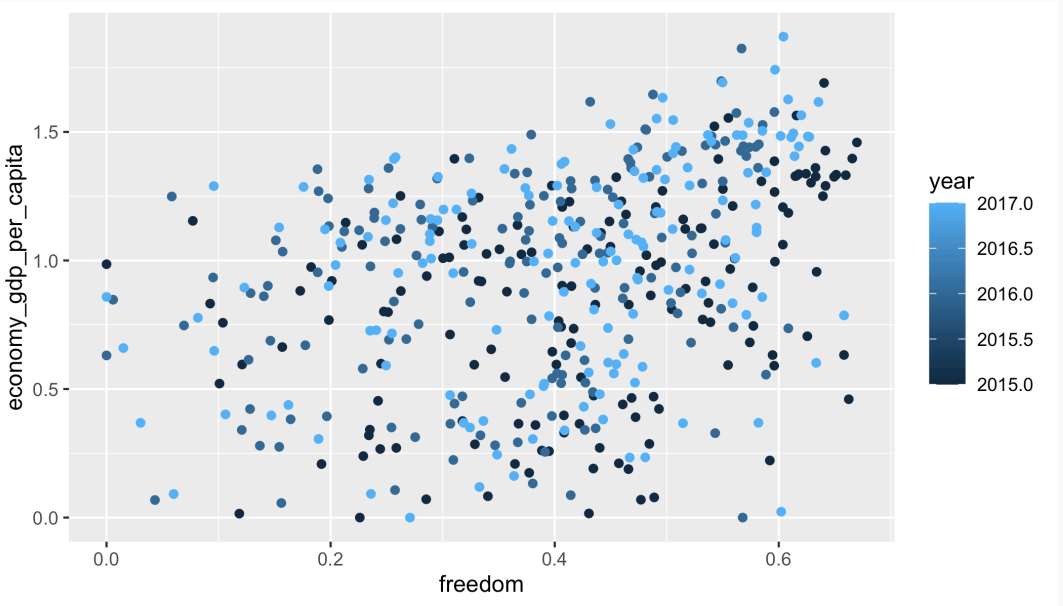
Exploratory Analysis

Exercise 4: Map colors per year for the freedom and gdp plot we did before. Keep in mind the type of the variable `year`.

1-minute exercise

Solution:

```
whr_panel %>%
  ggplot(
    aes(
      x = freedom,
      y = economy_gdp_per_capita,
      color = year
    )
  ) +
  geom_point()
```



Exploratory Analysis

How do you think we could solve it?

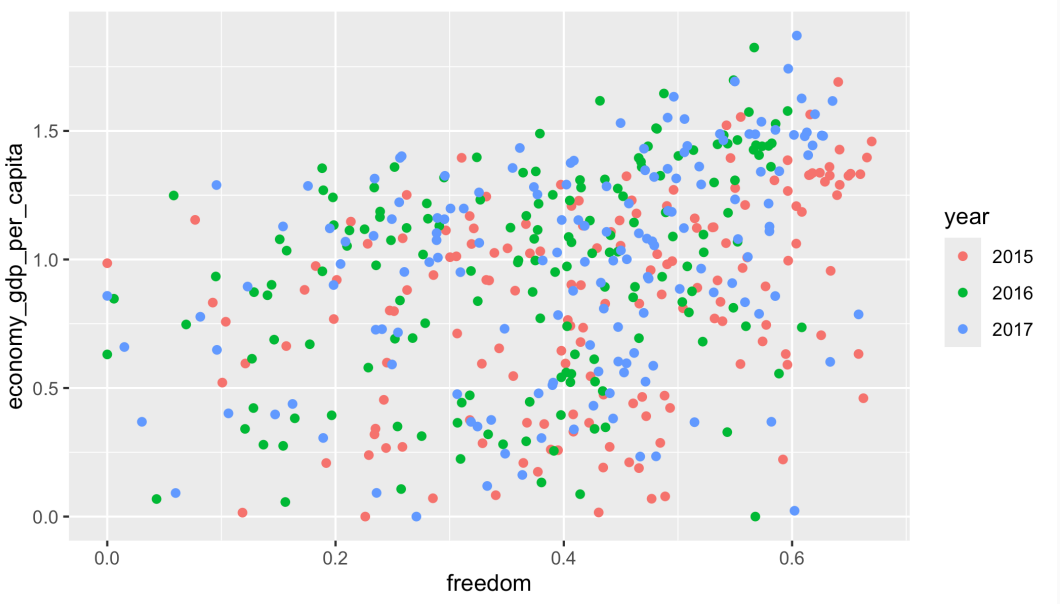
- Change the variable `year` as: `as.factor(year)`.

Exploratory Analysis

How do you think we could solve it?

- Change the variable `year` as: `as.factor(year)`.

```
whr_panel %>%
  ggplot(
    aes(
      x = freedom,
      y = economy_gdp_per_capita,
      color = as.factor(year)
    )
  ) +
  geom_point()+
  labs(color="year")
```



ggplot2: settings



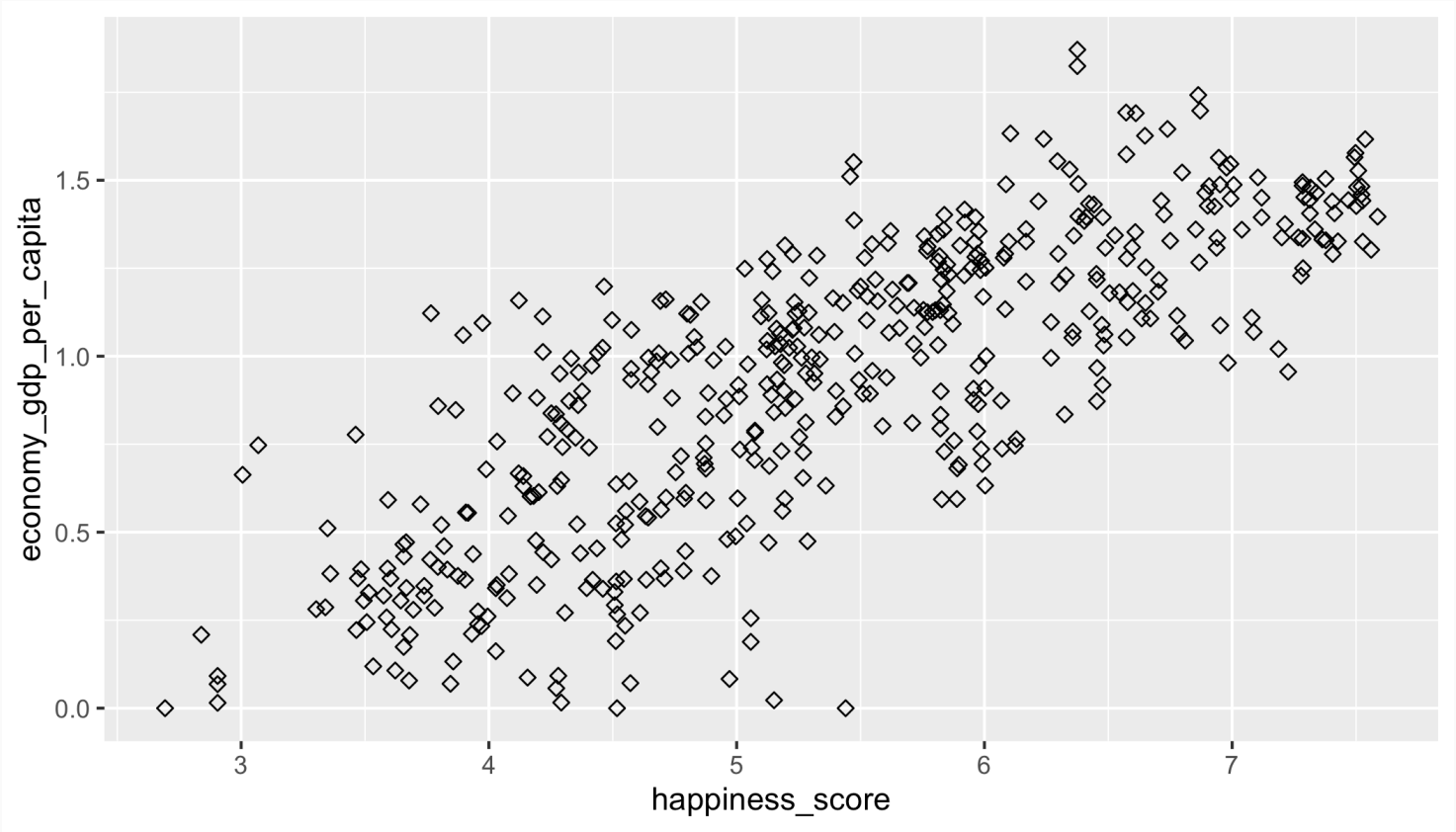
ggplot2: settings

Now, let's try to modify our plots. In the following slides, we are going to:

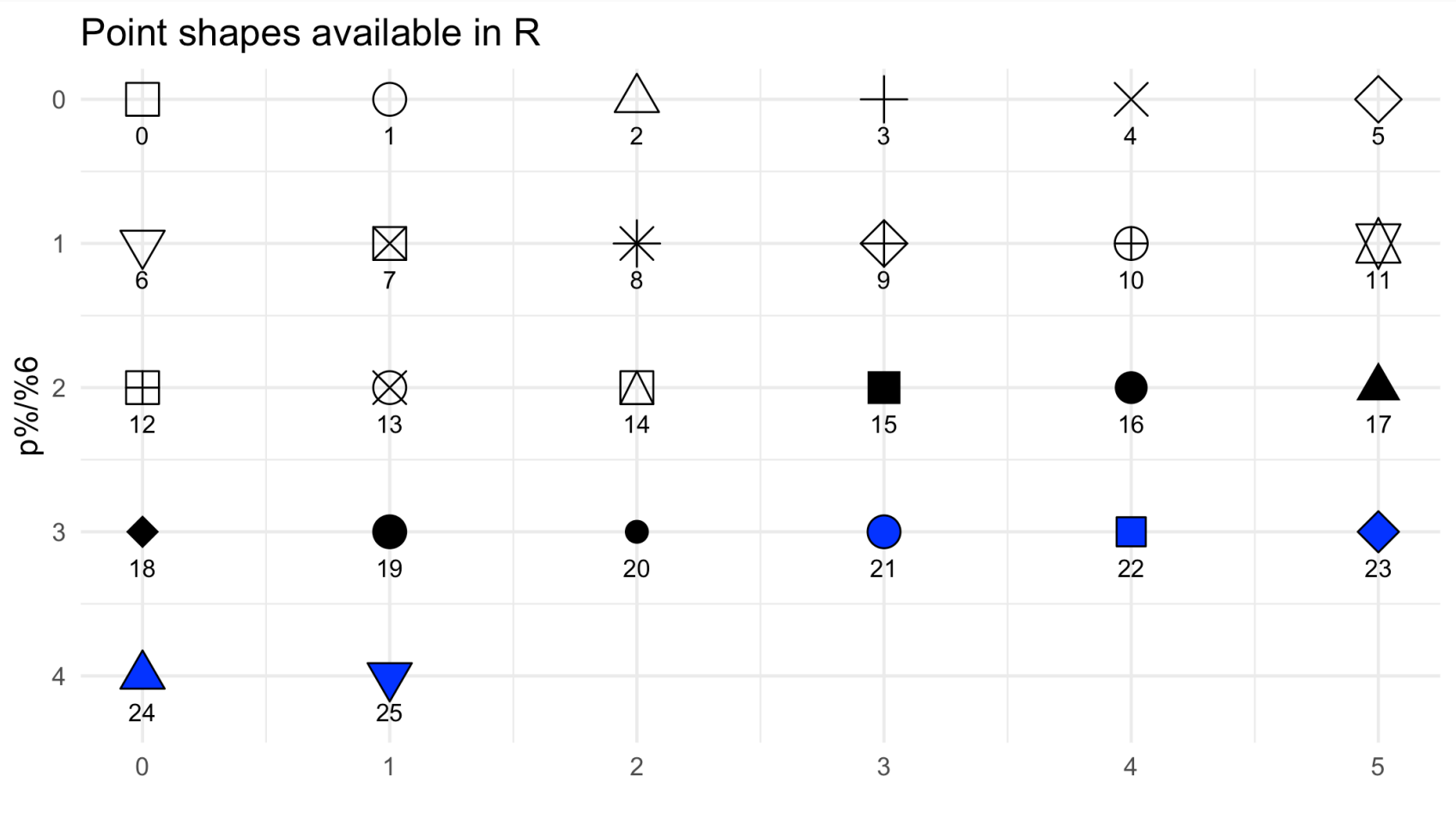
1. Change shapes.
2. Include more geoms.
3. Separate by regions.
4. Pipe and mutate before plotting.
5. Change scales.
6. Modify our theme.

ggplot2: shapes

```
whr_panel %>%
  ggplot(aes(x = happiness_score, y = economy_gdp_per_capita)) +
  geom_point(shape = 5)
```

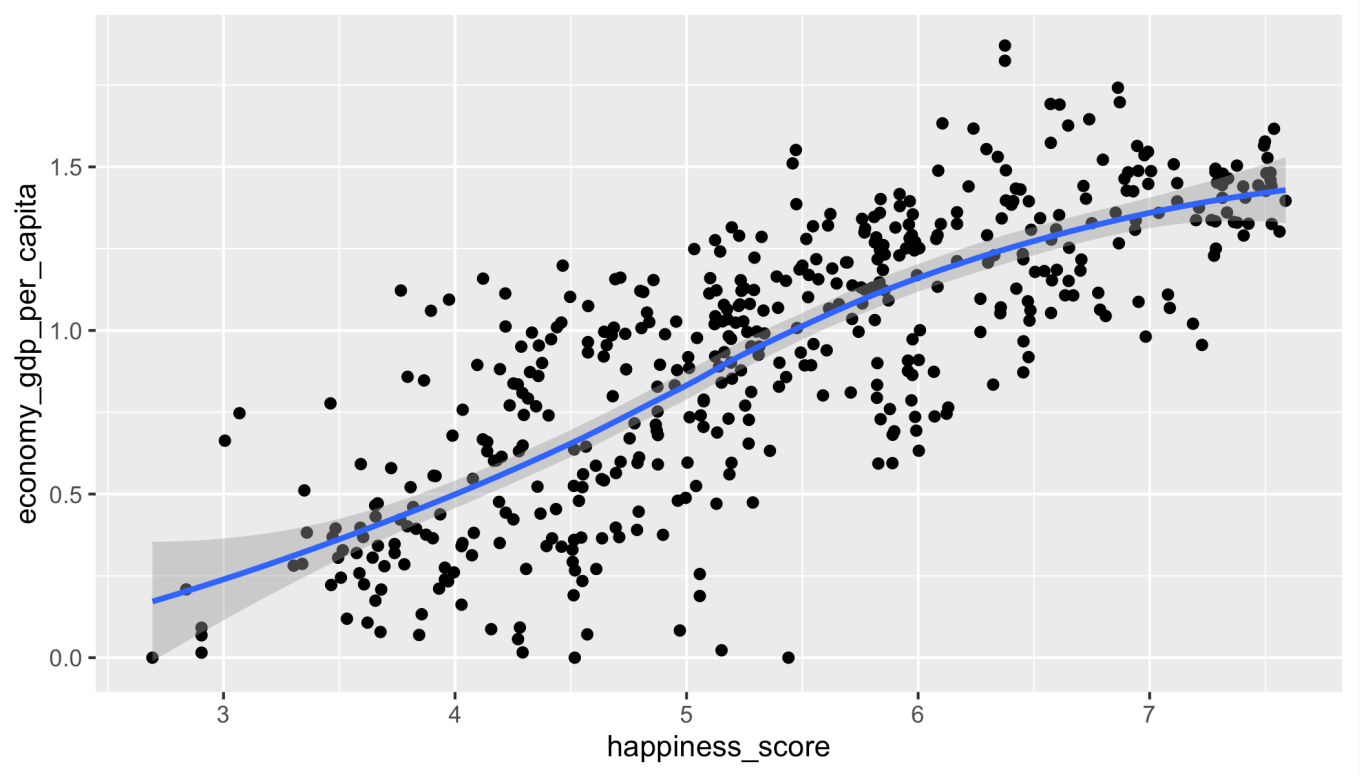


ggplot2: shapes



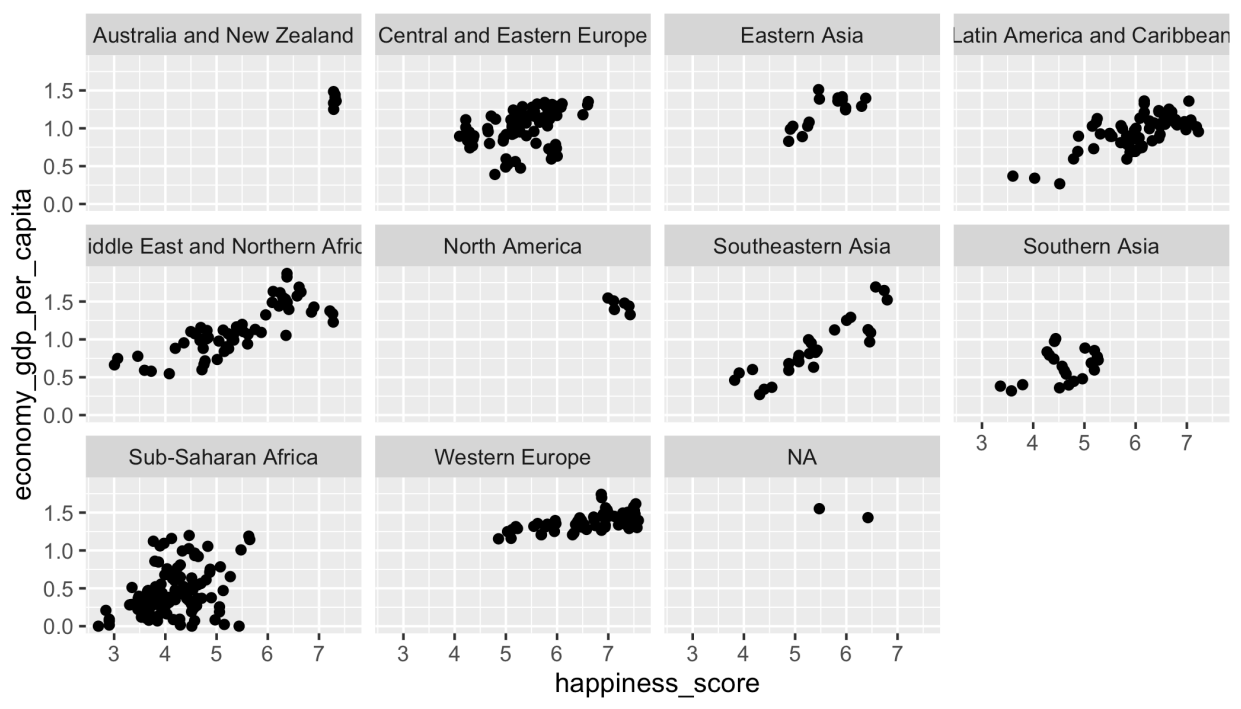
ggplot2: including more geoms

```
whr_panel %>%
  ggplot(aes(x = happiness_score,  y = economy_gdp_per_capita)) +
  geom_point() +
  geom_smooth()
```



ggplot2: Facets

```
whr_panel %>%
  ggplot(aes(x = happiness_score,  y = economy_gdp_per_capita)) +
  geom_point() +
  facet_wrap(~ region)
```



ggplot2: Colors and facets

Exercise 5: Use the last plot and add a color aesthetic per region.

1-minute exercise

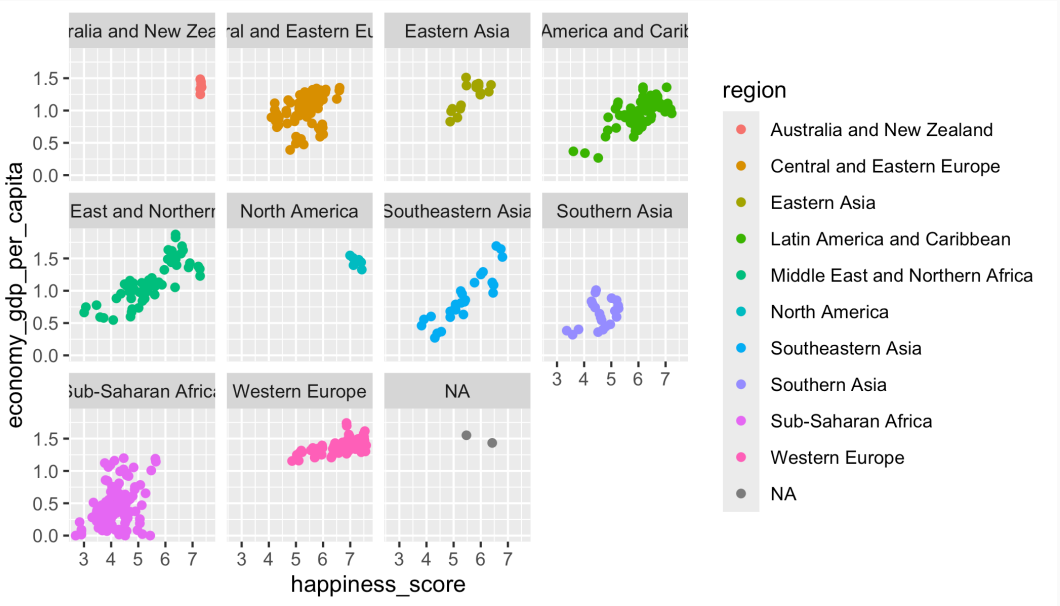
ggplot2: Colors and facets

Exercise 5: Use the last plot and add a color aesthetic per region.

1-minute exercise

Solution:

```
whr_panel %>%
  ggplot(
    aes(
      x = happiness_score,
      y = economy_gdp_per_capita,
      color = region
    )
  ) +
  geom_point() +
  facet_wrap(~ region)
```

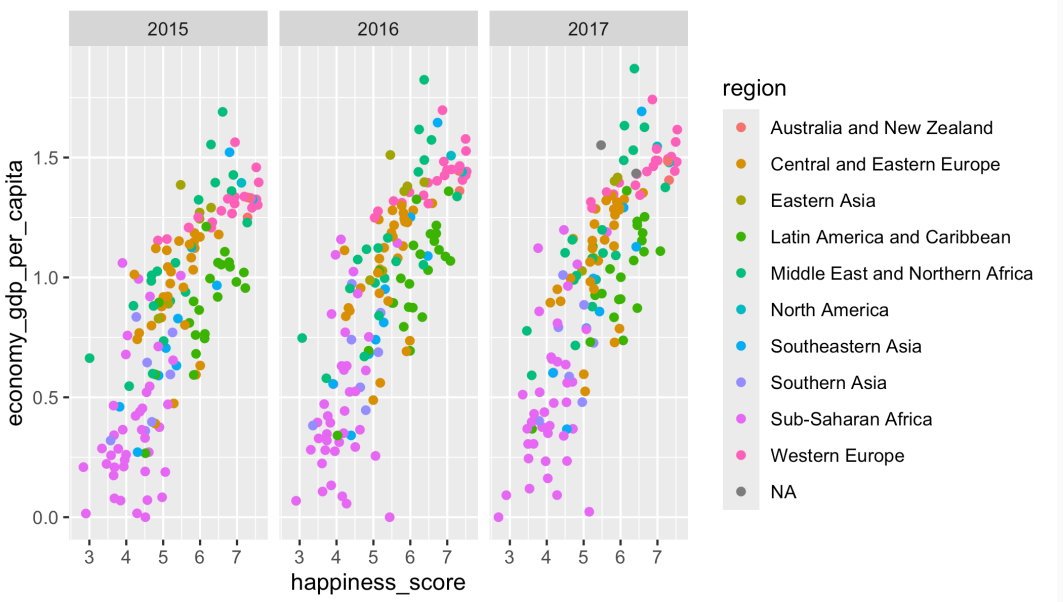


ggplot2: Colors and facets

Another type of facet is `facet_grid()`, which arranges plots by one or more variables while keeping a consistent y-axis across all panels. This is useful when comparing trends over time or across categories.

Here we facet the plot by `year`, so we can visually compare patterns in happiness and GDP over different years.

```
whr_panel %>%
  ggplot(
    aes(
      x = happiness_score,
      y = economy_gdp_per_capita,
      color = region
    )
  ) +
  geom_point() +
  facet_grid(~year)
```

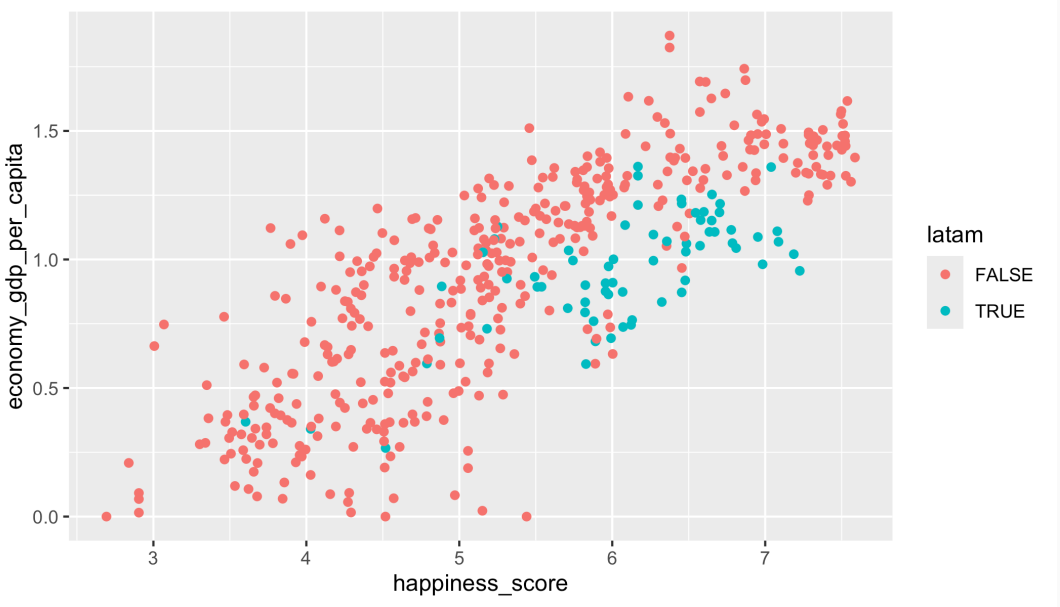


ggplot2: Pipe and mutate before plotting

Let's imagine now, that we would like to transform a variable before plotting.

```
whr_panel ← whr_panel %>%
  mutate(
    latam = region == "Latin America and Caribbean"
  )

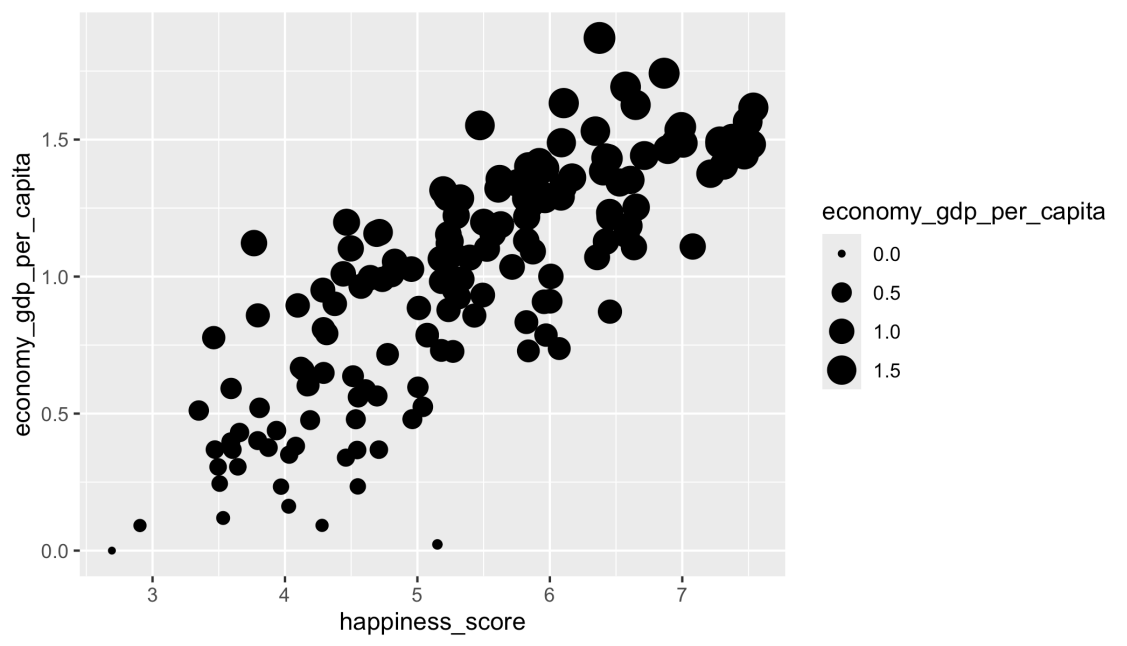
whr_panel %>%
  filter(
      !is.na(latam) # Make sure that we don't include r
  ) %>%
  ggplot(
    aes(
      x = happiness_score, y = economy_gdp_per_capita,
      color = latam)
  ) +
  geom_point()
```



ggplot2: geom's sizes

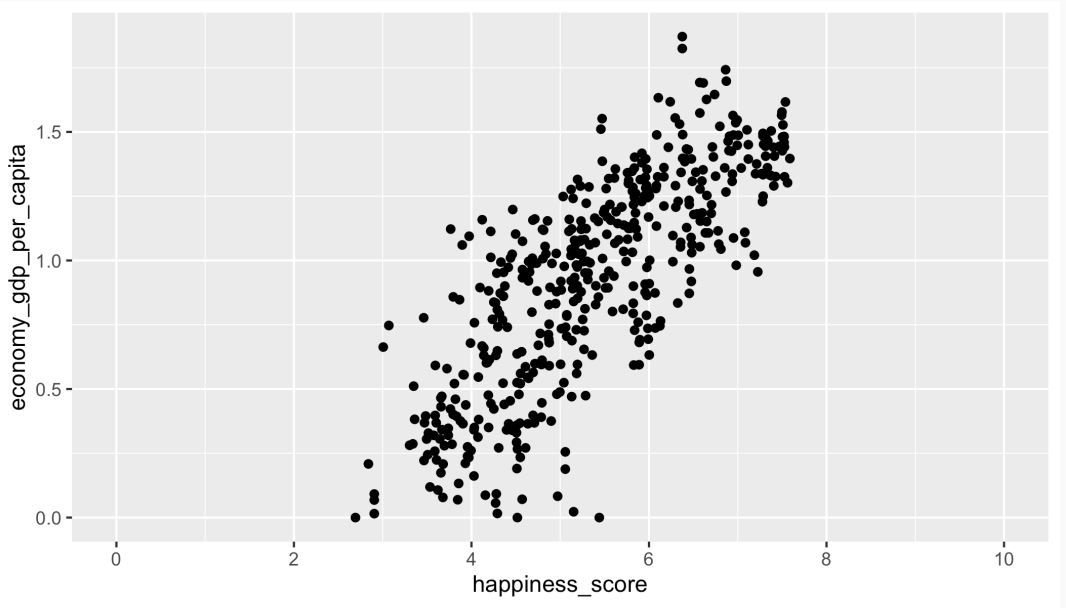
We can also specify the size of a geom, either by a variable or just a number.

```
whr_panel %>%
  filter(year == 2017) %>%
  ggplot(aes(x = happiness_score,  y = economy_gdp_per_capita)) +
  geom_point(aes(size = economy_gdp_per_capita))
```



ggplot2: Changing scales

```
ggplot(data = whr_panel,
       aes(x = happiness_score,
           y = economy_gdp_per_capita)) +
  geom_point()
```



]

ggplot2: Themes

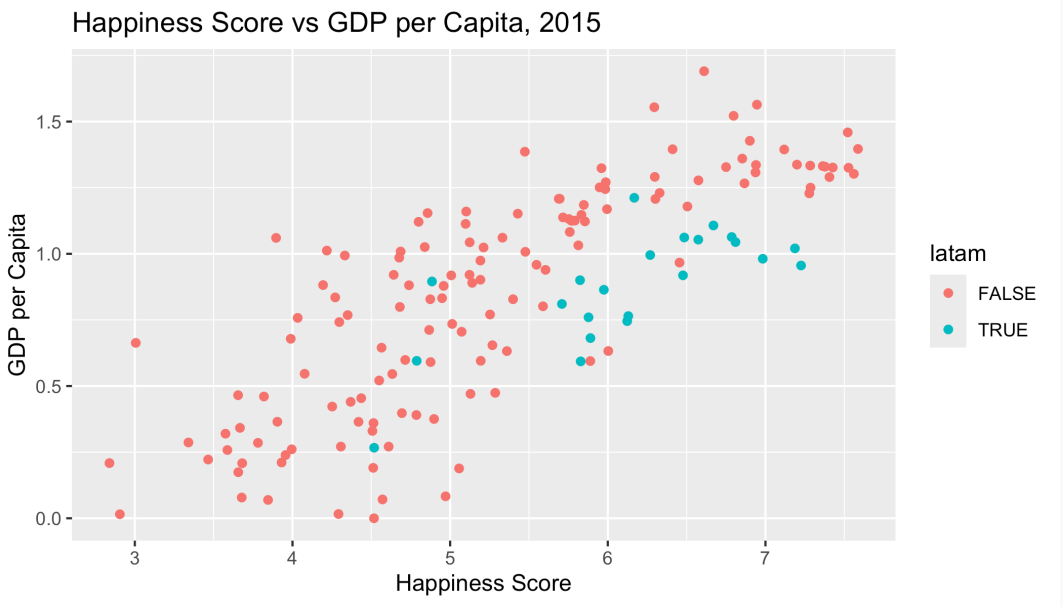
Let's go back to our plot with the `latam` dummy.

We are going to do the following to this plot:

1. Filter only for the year 2015.
2. Change our theme.
3. Add correct labels.
4. Add some annotations.
5. Modify our legends.

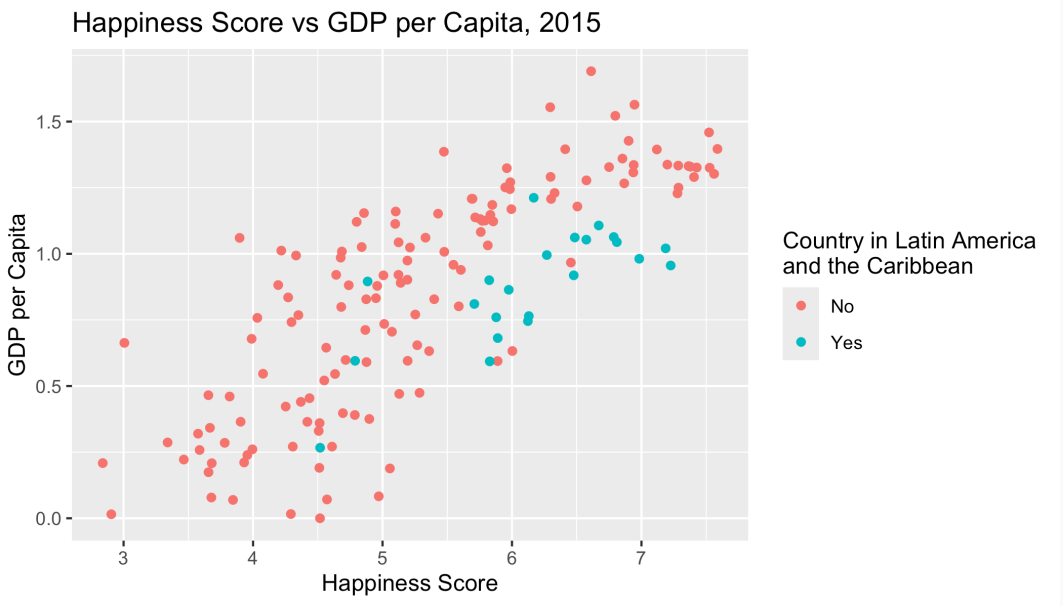
ggplot2: Labels

```
whr_panel %>%
  filter(
      !is.na(latam) # Make sure that we don't include r
  ) %>%
  filter(year == 2015) %>%
  ggplot(aes(x = happiness_score,
             y = economy_gdp_per_capita,
             color = latam)) +
  geom_point() +
  labs(
    x = "Happiness Score",
    y = "GDP per Capita",
    title = "Happiness Score vs GDP per Capita, 2015"
  )
```



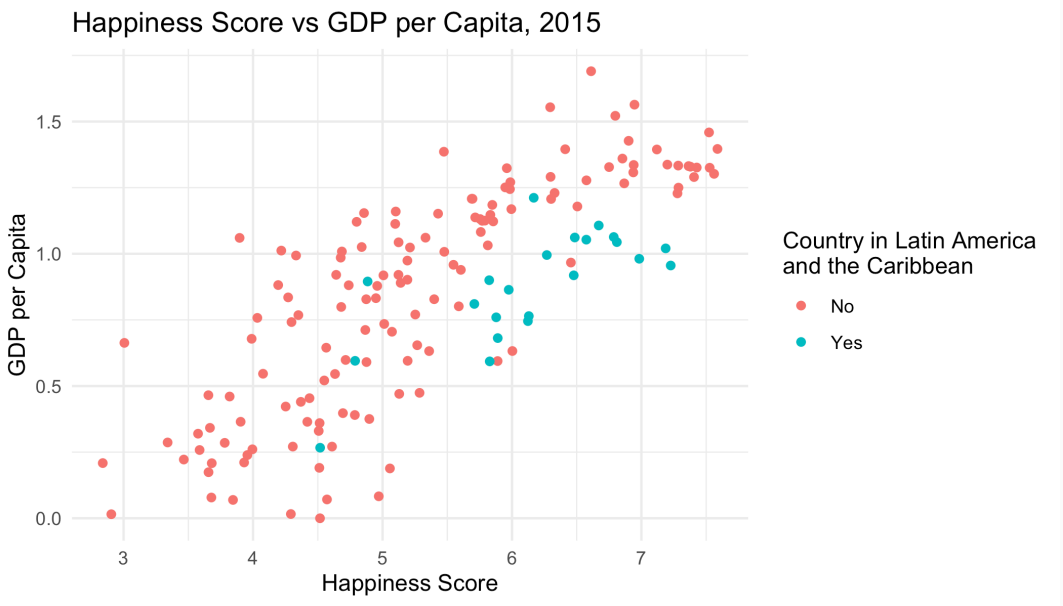
ggplot2: Legends

```
whr_panel %>%
  filter(
      !is.na(latam) # Make sure that we don't include r
  ) %>%
  filter(year == 2015) %>%
  ggplot(aes(x = happiness_score,
             y = economy_gdp_per_capita,
             color = latam)) +
  geom_point() +
  scale_color_discrete(labels = c("No", "Yes")) +
  labs(
    x = "Happiness Score",
    y = "GDP per Capita",
    color = "Country in Latin America\nand the Caribbea
    title = "Happiness Score vs GDP per Capita, 2015"
  )
```



ggplot2: Themes

```
whr_panel %>%
  filter(
      !is.na(latam) # Make sure that we don't include r
  ) %>%
  filter(year == 2015) %>%
  ggplot(aes(x = happiness_score,
             y = economy_gdp_per_capita,
             color = latam)) +
  geom_point() +
  scale_color_discrete(labels = c("No", "Yes")) +
  labs(
    x = "Happiness Score",
    y = "GDP per Capita",
    color = "Country in Latin America\nand the Caribbea
    title = "Happiness Score vs GDP per Capita, 2015"
  ) +
  theme_minimal()
```



ggplot2: Themes

The `theme()` function allows you to modify each aspect of your plot. Some arguments are:

```
theme(
    # Chart Title and text labels
    plot.title = element_text(color, size, face),
    # Legend title font color size and face
    legend.title = element_text(color, size, face),
    # Legend title alignment. Number from 0 (left) to 1 (right)
    legend.title.align = NULL,
    # Text label font color size and face
    legend.text = element_text(color, size, face),
    # Text label alignment. Number from 0 (left) to 1 (right)
    legend.text.align = NULL,
    # Axis label font color size and face
    axis.text = element_text(color, size,face),
    # Remove the minor grid lines from a graph
    panel.gid.minor = element_blank()
)
```

More about these modification can be found here

ggplot2: Color palettes

We can also add color palettes using other packages such as: `RColorBrewer, viridis` or funny ones like the `wesanderson` package. So, let's add new colors.

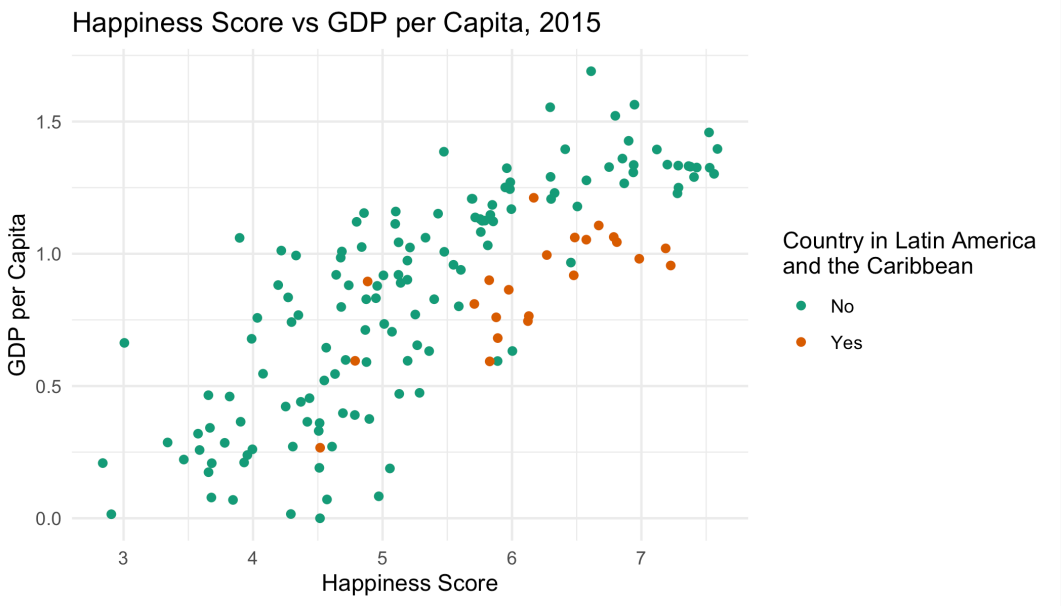
- First, install the `RColorBrewer` package.

```
# install.packages("RColorBrewer")
library(RColorBrewer)
```

- Let's add `scale_color_brewer(palette = "Dark2")` to our ggplot.

ggplot2: Color palettes

```
whr_panel %>%
  filter(
      !is.na(latam) # Make sure that we don't include r
  ) %>%
  filter(year == 2015) %>%
  ggplot(aes(x = happiness_score,
             y = economy_gdp_per_capita,
             color = latam)) +
  geom_point() +
  scale_color_brewer(palette = "Dark2",
                     labels = c("No", "Yes")) +
  labs(
    x = "Happiness Score",
    y = "GDP per Capita",
    color = "Country in Latin America\nand the Caribbea
    title = "Happiness Score vs GDP per Capita, 2015"
  ) +
  theme_minimal()
```



Appendix

Appendix - References and recommendations

- ggplot tricks:
  - Tricks and Secrets for Beautiful Plots in R by Cédric Scherer: https://github.com/z3tt/outlierconf2021
- Websites:
  - Interactive stuff : http://www.htmlwidgets.org/
  - The R Graph Gallery: https://www.r-graph-gallery.com/
  - Gpplot official site: http://ggplot2.tidyverse.org/
- Online courses:
  - Johns Hopkins Exploratory Data Analysis at Coursera: https://www.coursera.org/learn/exploratory-data-analysis
- Books:
  - The grammar of graphics by Leland Wilkinson.
  - Beautiful Evidence by Edward Tufte.
  - R Graphics cook book by Winston Chang
  - R for Data Science by Hadley Wickham andGarrett Grolemund

Takeaways and next session

Today:

- `cor()` measures linear association.
- `lm()` estimates regression coefficients; `summary()` helps interpret them.
- `modelsummary()` compares models in a readable default table.
- `ggplot() + aes() + geom_*()` is the core plotting pattern.

Next session: Further discuss various kinds of regression in R, and more advanced reporting.